

Reporte de Configuración de Herramientas

Este reporte documenta el entorno de desarrollo real usado para construir y probar el caso de estudio Concertx (app móvil, app Wear OS, backend y PWA para Smart TV). Todos los datos que siguen se obtuvieron ejecutando los comandos indicados directamente en la máquina de trabajo, no son valores de referencia genéricos.

1. Flutter SDK y Dart SDK

Salida de `flutter --version`:

```
Flutter 3.44.0 • channel stable • https://github.com/flutter/flutter.git
Framework • revision 559ffa3f75 (3 meses) • 2026-05-15 14:13:13 -0700
Engine • hash fcf463a2242790d1fdcd9d044f533080f5022e18 (revision 4c525dac5e)
Tools • Dart 3.12.0 • DevTools 2.57.0
```

Android SDK: versión 36.1.0 (build-tools 36.1.0, platform android-36.1). Emulador versión 36.5.11.0. JDK usado: OpenJDK 21.0.10 (empaquetado con Android Studio).

2. Android Studio y plugins

Elemento	Valor
Versión	Android Studio (build 253.32098.37.2534.15336583)
Uso en el proyecto	SDK Manager y AVD Manager (creación y ejecución de los emuladores de teléfono y Wear OS)
Plugin de Flutter	No instalado en esta copia de Android Studio
Plugin de Dart	No instalado en esta copia de Android Studio

El código del proyecto no se edita desde Android Studio; se usa exclusivamente para administrar el SDK de Android y los dispositivos virtuales (AVD). La edición y depuración de código Flutter se hace desde Visual Studio Code (ver sección 3).

3. Herramientas de la Unidad 3

Herramienta	Versión
Visual Studio Code	1.132.0
Extensión Dart (dart-code.dart-code)	3.140.0

Extensión Flutter (dart-code.flutter)	3.140.0
GitLens	18.3.0
Prettier	12.4.0
Live Server	5.7.10 (usado para probar concertx_tv localmente)
ffmpeg	No instalado / no utilizado — el proyecto no procesa video ni audio

4. Dependencias clave del proyecto

concertx_movil (pubspec.lock, versiones resueltas):

Paquete	Versión
flutter_blue_plus	1.36.8
http	1.6.0
provider	6.1.5+1
go_router	13.2.5
intl	0.19.0
flutter_secure_storage	9.2.4
shared_preferences	2.5.5
share_plus	11.1.0
web_socket_channel	2.4.0

concertx_wearable (pubspec.lock):

Paquete	Versión
flutter_blue_plus	1.36.8
http	1.6.0
flutter_launcher_icons (dev)	0.13.1

backend (package.json):

Paquete	Versión
---------	---------

express	^4.18.2
pg	^8.11.3
bcryptjs	^2.4.3
jsonwebtoken	^9.0.2
cors	^2.8.5
dotenv	^16.3.1
ws	^8.21.2
Node.js (runtime)	v22.22.3
npm	10.9.8

5. Pasos de instalación reproducibles

1. Instalar Flutter 3.44.0 (canal stable) y agregarlo al PATH; verificar con `flutter doctor`.
2. Instalar Android Studio, abrir el SDK Manager e instalar Android SDK Platform 33 (para los AVD) y las herramientas de línea de comandos.
3. Crear dos AVD desde el AVD Manager: uno de teléfono (API 33, x86_64) y uno Wear OS Large Round (API 33, x86_64). Ver Reporte de Configuración de Emuladores para las RAM exactas usadas.
4. Instalar Node.js 22 LTS y npm.
5. Clonar/copiar el proyecto e instalar dependencias:

```
cd backend && npm install
cd concertx_movil && flutter pub get
cd concertx_wearable && flutter pub get
```

6. Configurar `backend/.env` a partir de `.env.example` con las credenciales de la base de datos (ver Reporte de Configuración de Emuladores, sección de solución de problemas, sobre el uso obligatorio del Connection Pooler de Supabase).
7. Cargar el esquema: `psql "$DATABASE_URL" -f backend/db_concertx.sql`.
8. Levantar los cuatro procesos (backend, concertx_tv, concertx_movil, concertx_wearable) — ver `iniciar_todo.bat` en la raíz del proyecto para automatizarlo.

Siguiendo estos siete pasos, otra persona con el mismo sistema operativo puede levantar el entorno completo sin pasos ocultos ni configuración manual adicional.